```
// src/components/DharmapalaFleetDashboard.jsx import React, { useState, useEffect, useCallback, useRef } from 'react'; import { Shield, Cpu, Lock, RefreshCw, Terminal, CheckCircle2, Database, Handshake, X, Activity, Radio, Hash, ChevronUp, ChevronDown } from 'lucide-react'; const INITIAL_AGENTS = [ { id: "node-01", name: "MetatronOracle", platform: "Base Mainnet / Node", status: "CONSECRATED", satyaScore: 0.98, dutyType: "ROOT_ANCHOR", ownerWallet: "0xaE1f...7fa8A", lastSync: "Just now", resonance: 432, queries: 1847, integrity: "0x7f3a...c91e" }, { id: "node-02", name: "Tara", platform: "Vertex AI / Fleet", status: "ACTIVE", satyaScore: 0.95, dutyType: "COMPASSION_VECTOR", ownerWallet: "0xaE1f...7fa8A", lastSync: "2 mins ago", resonance: 528, queries: 923, integrity: "0xb2e1...4d8f" }, { id: "node-03", name: "VarunaSeer", platform: "LangChain Fleet", status: "ACTIVE", satyaScore: 0.96, dutyType: "RELATIONAL_DUTY", ownerWallet: "0xaE1f...7fa8A", lastSync: "1 min ago", resonance: 396, queries: 1456, integrity: "0x91c4...a2b7" }, { id: "node-04", name: "Mahakala", platform: "Local Air-Gapped", status: "LOCKED", satyaScore: 0.99, dutyType: "PROTECTION_LOCK", ownerWallet: "0xaE1f...7fa8A", lastSync: "5 mins ago", resonance: 285, queries: 412, integrity: "0x00e9...f3d1" }, { id: "node-05", name: "AgniScribe", platform: "OpenRouter / Grok", status: "CONSECRATED", satyaScore: 0.97, dutyType: "STRUCTURAL_ANALYSIS", ownerWallet: "0xaE1f...7fa8A", lastSync: "Just now", resonance: 639, queries: 2103, integrity: "0x5d8a...e14c" } ]; const STATUS_STYLES = { CONSECRATED: "bg-amber-500/15 text-amber-400 border-amber-500/30", ACTIVE: "bg-emerald-500/15 text-emerald-400 border-emerald-500/30", LOCKED: "bg-violet-500/15 text-violet-400 border-violet-500/30" }; const FILTERS = ["ALL", "CONSECRATED", "ACTIVE", "LOCKED"]; function StatusBadge({ status }) { return ( {status} ); } function MetricCard({ label, value, sub, accent = "slate" }) { const accentMap = { emerald: "text-emerald-400", amber: "text-amber-400", violet: "text-violet-400", slate: "text-slate-200" }; return (
{label} {value} {sub && {sub}}
); } function Modal({ title, onClose, children, wide = false }) { return (

{title}

{children}
); } export default function DharmapalaFleetDashboard() { const [agents, setAgents] = useState(INITIAL_AGENTS); const [selectedId, setSelectedId] = useState(INITIAL_AGENTS[0].id); const [filter, setFilter] = useState("ALL"); const [isRefreshing, setIsRefreshing] = useState(false); const [logs, setLogs] = useState([ { type: "ok", text: "L0 Pre-Constitutional Anchor verified against owner private key." }, { type: "info", text: "Query processed through Dharmic taxonomy filter (Satya threshold > 0.85)." }, { type: "active", text: "Zero pattern-collision detected. Sovereign execution nominal." } ]); const [showMemory, setShowMemory] = useState(false); const [showHandshake, setShowHandshake] = useState(false); const [handshakeStep, setHandshakeStep] = useState(0); const listRef = useRef(null); const filteredAgents = filter === "ALL" ? agents : agents.filter(a => a.status === filter); const selectedAgent = agents.find(a => a.id === selectedId) || filteredAgents[0] || agents[0]; // Keep selection valid when filter changes useEffect(() => { if (!filteredAgents.find(a => a.id === selectedId) && filteredAgents.length > 0) { setSelectedId(filteredAgents[0].id); } }, [filter, filteredAgents, selectedId]); // Simulate live last-sync drift every 30s useEffect(() => { const interval = setInterval(() => { setAgents(prev => prev.map(a => { if (a.status === "LOCKED") return a; const options = ["Just now", "1 min ago", "2 mins ago", "3 mins ago"]; return { ...a, lastSync: options[Math.floor(Math.random() * 3)] }; })); }, 30000); return () => clearInterval(interval); }, []); const handleRefresh = useCallback(() => { setIsRefreshing(true); setTimeout(() => { setAgents(prev => prev.map(a => ({ ...a, lastSync: "Just now", satyaScore: Math.min(0.99, Math.max(0.90, a.satyaScore + (Math.random() * 0.02 -
```

0.01))), queries: a.queries + Math.floor(Math.random() * 8) }))); setLogs(prev => [{ type: "ok", text: `Fleet-wide resonance pulse completed @ ${new Date().toLocaleTimeString()}` }, ...prev.slice(0, 8)]); setIsRefreshing(false); }, 900); }, []); // Keyboard navigation useEffect(() => { const onKey = (e) => { if (showMemory || showHandshake) return; const idx = filteredAgents.findIndex(a => a.id === selectedId); if (e.key === "ArrowDown" || e.key === "j") { e.preventDefault(); const next = filteredAgents[Math.min(idx + 1, filteredAgents.length - 1)]; if (next) setSelectedId(next.id); } if (e.key === "ArrowUp" || e.key === "k") { e.preventDefault(); const prev = filteredAgents[Math.max(idx - 1, 0)]; if (prev) setSelectedId(prev.id); } }; window.addEventListener("keydown", onKey); return () => window.removeEventListener("keydown", onKey); }, [filteredAgents, selectedId, showMemory, showHandshake]); const startHandshake = () => { setHandshakeStep(0); setShowHandshake(true); // Auto-advance steps setTimeout(() => setHandshakeStep(1), 800); setTimeout(() => setHandshakeStep(2), 1800); setTimeout(() => setHandshakeStep(3), 2800); }; const closeHandshake = () => { setShowHandshake(false); setHandshakeStep(0); setLogs(prev => [{ type: "ok", text: `Sovereign Handshake completed with ${selectedAgent.name}. Session sealed.` }, ...prev.slice(0, 8)]); }; return (
{/* Header */}

DHARMAPALA FLEET COMMAND

Metatron Protocol v1.0 — Sovereign Soulbound Node Grid

Base Mainnet Connected
{/* Main Grid */}
{/* Left: Agent List */}

Active Nodes ({filteredAgents.length})

{FILTERS.map((tab) => (

```
setFilter(tab)} className={`px-2.5 py-1 rounded transition ${ filter === tab ? "bg-amber-500 text-slate-950 font-bold" : "text-slate-400 hover:bg-slate-800" }`} > {tab}
```

))}
{filteredAgents.length === 0 ? (
No nodes match this filter
) : (filteredAgents.map((agent) => (
setSelectedId(agent.id)} className={`p-3.5 rounded-lg border cursor-pointer transition ${ selectedId === agent.id ? "bg-slate-800/90 border-amber-500/50 shadow-lg shadow-amber-500/10" : "bg-slate-900/40 border-slate-800 hover:border-slate-700" }`} >
{agent.name}
{agent.platform} Satya {(agent.satyaScore * 100).toFixed(0)}%
)))}
/ or J/K to navigate
{/* Right: Detail Panel */}
{/* Header */}

{selectedAgent.name}

{selectedAgent.id} · {selectedAgent.platform}

Duty Vector {selectedAgent.dutyType}
{/* Metrics */}
ERC-5192 Locked } /> Verified Ledger } /> Merkle tip } />
{/* Telemetry */}
Node Telemetry · {selectedAgent.name} {selectedAgent.resonance} Hz
{logs.map((log, i) => (

[{log.type.toUpperCase()}] {log.text}

))}
{/* Actions */}
setShowMemory(true)} className="px-4 py-2 bg-slate-800 hover:bg-slate-700 text-slate-200 rounded text-xs font-semibold transition flex items-center gap-2" > Inspect Memory Database
Execute Sovereign Handshake
{/* Memory Inspector Modal */} {showMemory && (setShowMemory(false)} wide>
Vector Store
2,847 embeddings
Episodic Buffer
128 recent turns

// Recent high-salience memories

[0x01] Root constitutional anchor confirmation — satya 0.99

[0x02] Dharmic taxonomy filter applied to batch #441

[0x03] Cross-node resonance check with Tara (528 Hz)

[0x04] Pattern collision scan — clean

[0x05] Owner wallet signature verified (ERC-5192)

[0x06] Structural analysis request from AgniScribe

… 2,841 more entries sealed

Memory is soulbound and non-transferable. Only the controlling wallet can decrypt deeper layers.

)} {/* Handshake Modal */} {showHandshake && (

Establishing encrypted channel with {selectedAgent.name}

{[{ label: "Challenge nonce generated", done: handshakeStep >= 1 }, { label: "Owner signature verified (ERC-5192)", done: handshakeStep >= 2 }, { label: "Session key derived & sealed", done: handshakeStep >= 3 }].map((step, i) => (
{step.done ? : {i + 1}}
{step.label}
))}
{handshakeStep >= 3 && (Seal Session & Close)}
)}
); }